

LogBotAI – Dokumentation

LogBotAI är en React-applikation som gör det möjligt för användare att ladda upp .docx-filer, extrahera textinnehållet, och sedan interagera med en chatbot som använder den inbäddade informationen från de uppladdade filerna. Appen utvecklades med hjälp av Visual Studio Code som utvecklingsmiljö. Jag använde React för frontend och Node.js för backend. Google Cloud Platform användes för att lagra och publicera applikationen online. För backend-koden använde jag GitHub som en värdtjänst, och Render.com användes för webbtjänsten som kopplar samman applikationens olika delar. Applikationen är uppbyggd kring två huvudfunktioner: filhantering och chatbot-interaktion.

1. Filhantering och uppladdning

Applikationen erbjuder användaren två sätt att ladda upp dokument: antingen genom att dra och släppa filer direkt i gränssnittet eller genom att välja filer via en filväljare. När användaren laddar upp filer filtrerar systemet automatiskt bort alla filer som inte har .docx-formatet. Dessutom är det en maxgräns på 50 filer som kan laddas upp samtidigt. Om denna gräns överskrids visas ett meddelande för att informera användaren om att bara de första 50 filerna kommer att accepteras.

Efter att filerna valts ut, använder applikationen FileReader-API:t för att läsa in filerna. Varje fil omvandlas till ett binärt format (ArrayBuffer), vilket sedan bearbetas av biblioteket mammoth för att extrahera rå text från .docx-filerna. Denna text lagras i en lokal tillståndsvariabel (filesContents) och är nu redo att skickas till backend för inbäddning.

När användaren trycker på “Ladda upp” skickas de extraherade filinnehållen till backend, där de bearbetas via en API-endpoint som heter /textToEmbedd. Backend-systemet tar emot texterna, skapar inbäddningar av dem och skickar tillbaka dessa till frontend-applikationen. Inbäddningarna lagras i applikationens tillstånd och används senare för att svara på användarens frågor i chatbot-sektionen.

2. Chatbot-interaktion

Efter att filerna har laddats upp och inbäddningar genererats, kan användaren börja ställa frågor till en inbyggd chatbot. Chatbotens uppgift är att besvara frågor baserade på det inbäddade innehållet från filerna.

När användaren skriver in en fråga i textfältet och skickar den, skickar applikationen frågan tillsammans med de tidigare genererade inbäddningarna till backend via en annan API-endpoint kallad /chat. Backend bearbetar frågan, jämför den med inbäddningarna från de uppladdade dokumenten, och returnerar ett svar som sedan visas i chatfönstret för användaren.

3. Användargränssnitt

Gränssnittet är uppdelat i två huvuddelar: filhantering och chatbot-sektion.

Filhanteringssektionen inkluderar både drag-och-släpp-funktionalitet och en knapp för att välja filer manuellt. När filer har laddats upp, visas namnen på de valda filerna i gränssnittet, och användaren kan trycka på en “Ladda upp”-knapp för att skicka filerna till backend. Under uppladdningen visas en laddningsindikator för att ge feedback till användaren om att uppladdningen pågår. Om uppladdningen lyckas, visas en bekräftelse, och användaren kan välja att “Nollställa” processen för att börja om från början.

Chatbot-sektionen visas endast efter att filer har laddats upp. Här finns ett chattfönster där användarens frågor och chatbotens svar visas i realtid. Användaren kan ställa en fråga genom att skriva i textfältet och skicka den, varpå svaret visas direkt under frågan i fönstret.

4. Backend-interaktion och felhantering

Applikationen är beroende av en backend som hanterar inbäddning och frågeanalys. Den skickar två huvudsakliga förfrågningar till backend: en för att generera inbäddningar från textinnehållet i de uppladdade filerna, och en annan för att skicka frågor och ta emot svar från chatboten.

Denna backend är byggd med Node.js och Express och ansvarar för två huvudfunktioner i applikationen: att skapa inbäddningar av text och att hantera frågor från en chatbot baserat på dessa inbäddningar. Den använder OpenAI:s modell för textinbäddningar samt en chattmodell för att generera svar. Nedan följer en förklaring av varje del av koden.

4.1. Textinbäddning via /textToEmbedd-endpoint

När en användare laddar upp en eller flera .docx-filer i frontend-applikationen, skickas innehållet från dessa filer till backend genom denna endpoint. Backend tar emot texterna och skapar inbäddningar med OpenAI:s modell text-embedding-ada-002. Här är de viktigaste stegen i processen:

- **Tokenräkning:** För att skapa inbäddningar är det viktigt att hålla sig inom en token-gräns (maximalt 8191 tokens per förfrågan). En token är en bit text som modellen bearbetar, och det kan vara ett ord, en del av ett ord, eller ett skiljetecken. Med hjälp av `encoding_for_model`-funktionen räknas antalet tokens i varje textbit.
- **Textdelning:** Om en text överskrider den maximala token-gränsen, delas texten upp i mindre delar. Detta görs genom att koda texten till tokens och sedan dela upp dem i bitar med högst 8191 tokens var. Varje bit konverteras sedan tillbaka till text och bearbetas individuellt.

- **Inbäddningsskapande:** Varje textdel skickas till OpenAI:s API där en inbäddning (en numerisk representation av textens innebörd) skapas. Dessa inbäddningar sparas sedan i en array som returneras till frontend-applikationen.

4.2. Chat-funktionalitet via /chat-endpoint

Efter att inbäddningar har skapats, kan användaren ställa frågor till en chatbot som baseras på de uppladdade filerna. Backend-processen för att hantera detta består av följande steg:

- **Frågeinbäddning:** När användaren skickar en fråga, skapar backend en inbäddning av själva frågan med hjälp av samma modell som användes för att skapa inbäddningarna för filinnehållet.
- **Likhetssökning:** Backend jämför frågans inbäddning med de tidigare sparade inbäddningarna från de uppladdade filerna för att hitta de mest liknande textinnehållen. Detta görs med en metod för att beräkna cosinuslikhet, vilket mäter hur lika två vektorer (inbäddningar) är. De fem mest liknande resultaten returneras.
- **Generering av svar:** När de mest relevanta textdelarna har identifierats, skickar backend dessa till OpenAI:s GPT-3.5-modell tillsammans med användarens fråga. Modellen genererar sedan ett svar baserat på dessa relevanta texter, som skickas tillbaka till frontend-applikationen där det visas för användaren.

4.3. Felhantering och loggning

Koden har även en del inbyggd felhantering. Om något går fel vid inbäddningsskapandet, som att en text inte är i rätt format, fångas detta och ett felmeddelande loggas. På samma sätt hanteras eventuella problem med att skapa inbäddningar eller svar från OpenAI:s API.

Länk till appen:

<https://logbotai-380524912014.europe-west1.run.app/> - GCP Google Cloud Platform för lagring och publicering